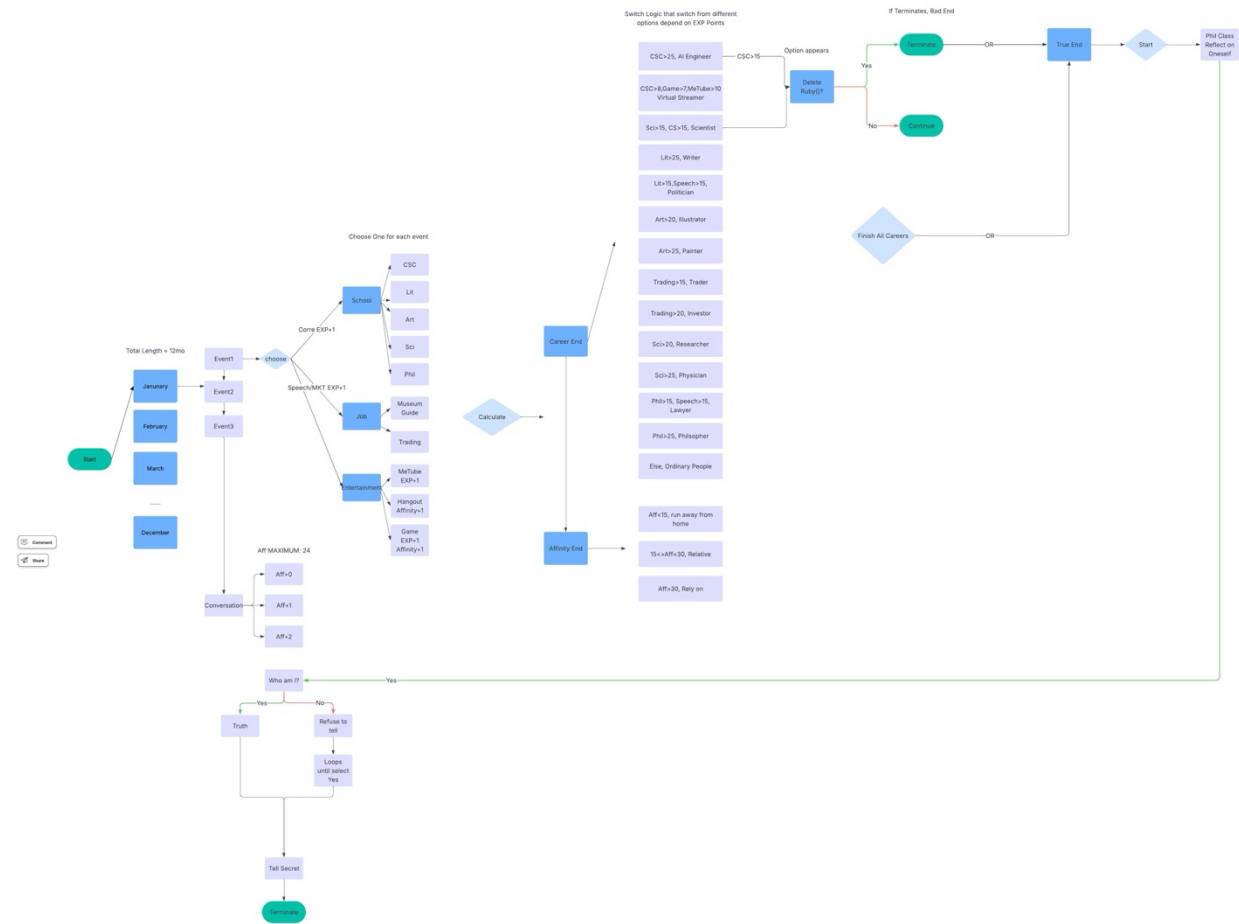




Here is an initial flow chart that we try to develop. We cut some features (such as the “who am I” part and the affinity system because of time constraint. Other than that, the game focuses on plot and GUI. Therefore, we add some interesting designs such as the TextEffect class which includes a method to print character one by one.

The game starts at the Main class, which initializes all the EXP variables. It then calls the StartView class which includes everything from initialization (the first scene) until the button selection page. At the end of the StartView class, it calls the ViewManager class which has all the buttons and the event after player clicks each button. After clicking buttons for 36 times in total to simulate the action to do in life, the program then goes back to the main class and calls

EndingView class that calculates the EXP and decides the ending. We make a separate main class to make the structure clearer and easier to debug if anything goes wrong.

The StartView class is composed of methods to find and set background image, mouse-click events, and plots. We use if-statements and incremental variable clickCount to push forward the plot (change designs) each time player clicks the mouse. At the last click (clickCount ==15), the program calls ViewManager and ends the StartView class. The createBackground method initializes the default background image.

The ViewManager class is the main body of Ruby. Since button designs are created in a separate ButtonView class to again make the structure easier to understand, we simply call the method and add the buttons here with text. The createmenuButton/addmenuButton methods also adjust the position of buttons to make them displayed in 4 rows.

The most important part in ViewManager is the handleActivitySelection method which includes all the events after player clicks the buttons, including a different background image, a line of text in the middle, and corresponding EXP points. We use switch and trackTurn to store the number of times player clicks the buttons (select events). If it reaches 36 times, the game terminates.

After ViewManager terminates, EndingView is called to display the career ending (or normal ending) tracked by EXP points. We do some value control to make sure the conditions in the if-statement does not violate each other. We also include a true ending that will operate when player has experienced all the career endings.

As mentioned above, the TextEffect class and ButtonView class are used to set corresponding designs. Because they are used in multiple places in Ruby, we put them separately that can be called easily by putting in the parameters.

The images are stored in the resources' directory and the sources of images are attached below in the appendix. All of them are found online.

Thank you so much for reviewing!

Appendix

//Home.jpeg

<https://unsplash.com/photos/white-and-brown-living-room-set-ztWpwTEx728>

//Home1.jpg

<https://www.decorpot.com/blog/guide-on-home-interior-colors>

//Futuristic.jpg

<https://wallpapersok.com/wallpapers/cyberpunk-city-lights-in-rain-ksyb2m5cdc66pa2a.html>

//CS.jpg

<https://cas.umw.edu/computerscience/>

//Literature.jpeg

<https://shakespearedocumented.folger.edu/resource/document/earliest-manuscript-copy-shakespeare-play-conflated-version-two-parts-henry-iv>

//Science.jpg

<https://arstechnica.com/science/2024/02/new-e-coli-strain-will-accelerate-evolution-of-the-genes-of-your-choice/>

//Philosophy.jpg

<https://flc.losrios.edu/academics/programs-and-majors/philosophy>

//MarketTrading.jpg

<https://www.nasdaq.com/market-activity/stock-market-holiday-schedule>

//MeTube.jpg

<https://www.vecteezy.com/vector-art/24081640-hud-video-player-futuristic-interface-web-design>

//HangOut.jpg

<https://www.ingka-centres.com/cs/newsroom/ingka-centres-launches-its-first-ikea-anchored-meeting-place-in-india/>

//Play.jpg

https://store.steampowered.com/app/1426210/_/